# Django Notes

## 1. Introduction to Django

### What is Django?

→ Django is a high level Python web framework that encourages rapid development and clean, pragmatic design.

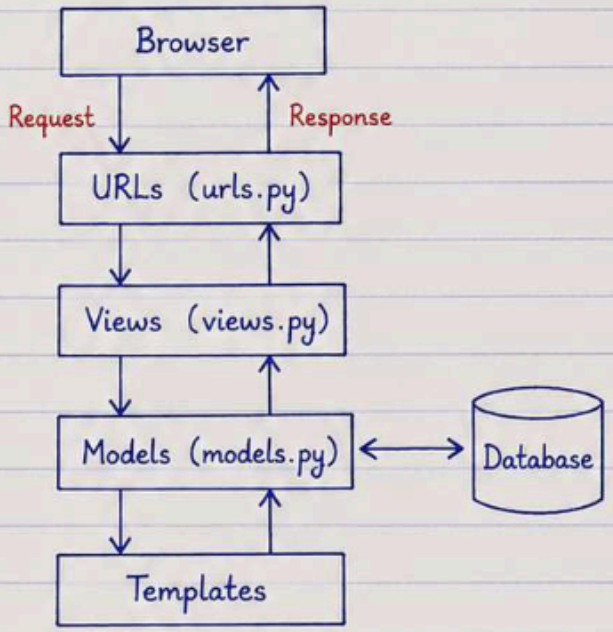
### Why Django?

→ Built-in admin panel
→ Secure by default
→ Scalable and maintainable
→ Follows DRY principle
→ Large community support

### Key Features

- ORM (Object Relational Mapping)
- MVT Architecture
- Built-in authentication
- Form handling and validation
- Template engine
- Middleware support
- Easy testing

### Django Architecture (MVT)




### How Django Works?

1. User sends request from browser.
2. URL patterns match the request.
3. View handles the request.
4. Model interacts with Database.
5. Template is rendered.
6. Response is sent back to browser.

### Install Django

```
pip install django
```

### Check Version

```
python -m django --version
```

# Django Notes

## 2. Django Project Structure

### Create a Project

```
django-admin startproject myproject
```

This command creates a Django project with the following structure.

### Project Structure

```
myproject/
    ├── manage.py
    └── myproject/
        ├── __init__.py
        ├── settings.py
        ├── urls.py
        ├── asgi.py
        └── wsgi.py
```

### Main Files

- **manage.py** – Command line utility to run project tasks.
- **settings.py** – Project configuration (Database, Installed apps, etc.)
- **urls.py** – URL declarations for the project.
- **asgi.py** – ASGI config for async applications.
- **wsgi.py** – WSGI config for deployment.
- **__init__.py** – Tells Python that this is a package.

### Run the Project

```
python manage.py runserver
```

→ Starts the development server.

Open in browser: http://127.0.0.1:8000/

### Summary

A Django project is a collection of settings, apps, and configuration that work together to build a web application.

# Django Notes

## 3. Django App & URL Routing

### Create an App

```
python manage.py startapp blog
```

This command creates a new app named 'blog' inside the project.

### Why Apps?

→ Helps in modular development
→ Each app has its own features
→ Easy to maintain and reuse
→ Can be added to any project

### URL Routing

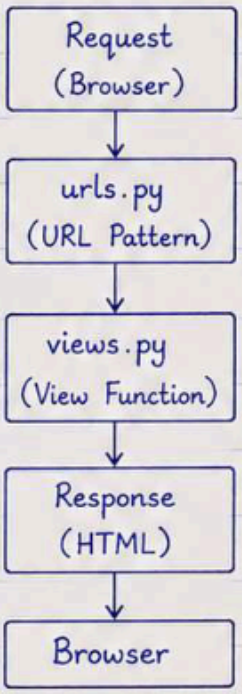
URLs (Uniform Resource locators) route requests to the appropriate view.

### Example :

```
# urls.py
from django.urls import path
from . import views

urlpatterns = [
    path('home/', views.home, name='home'),
    path('about/', views.about, name='about'),
]
```

### Request Flow




### Steps

1. User enters a URL in the browser.
2. Django matches the URL with urlpatterns in urls.py.
3. The corresponding view function is called.
4. View returns a response (HTML / JSON).
5. Response is sent back to the browser.

### Key Points

- URLs act as the entry point of any Django project.
- Each app can have its own urls.py file.
- Always include app urls in the project urls.py using include().

# 4. Models & ORM

## What is Model ?

Model is a Python class that represents a table in the database.

## Example :

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()
    email = models.EmailField()
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.name
```

## Common Field Types

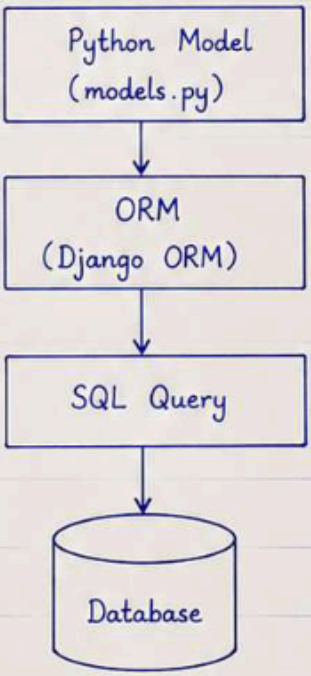
- CharField() – Text data
- IntegerField() – Integer numbers
- EmailField() – Email address
- DateField() – Date
- DateTimeField() – Date & Time
- BooleanField() – True / False
- ForeignKey() – Relationship
- ManyToManyField() – Many to Many

## Migrations

- Migrations are used to apply changes made in models to the database.

```
python manage.py makemigrations
python manage.py migrate
```

## ORM Flow




## Relationships

- OneToOneField() – One to One
- ForeignKey() – One to Many
- ManyToManyField() – Many to Many

# 5. Views & Templates

## What is a View?

A view is a Python function or class that takes a web request and returns a web response.

## Types of Views

- Function Based View (FBV)
- Class Based View (CBV)

## Function Based View (FBV)

```
from django.shortcuts import render

def home(request):
    data = { 'title' : 'Home Page' }
    return render(request, 'home.html', data)
```

## Template Rendering

Views pass data to templates.
Templates generate HTML.

Example Template (home.html)

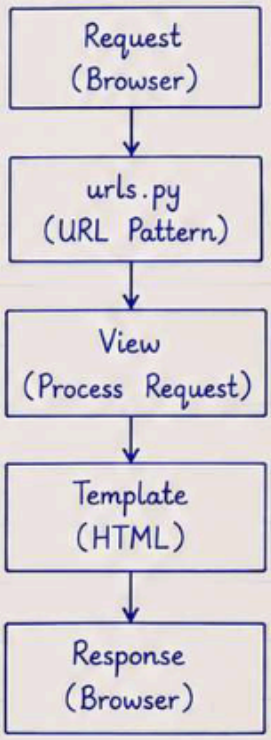
```
<h1>{{ title }}</h1>
<p>Welcome to Django!</p>
```

## Class Based View (CBV)

```
from django.views import View
from django.shortcuts import render

class HomeView(View):
    def get(self, request):
        data = { 'title' : 'Home Page' }
        return render(request, 'home.html', data)
```

## Flow




## Steps

1. User sends request from browser.
2. URLs match the request.
3. Corresponding view is called.
4. View processes the request.
5. View renders the template.
6. HTML is returned to the browser.

# 6. Forms & Admin Panel

## What is a Form ?

Forms are used to take input from users and validate the data.

## Django Forms

```
from django import forms

class ContactForm(forms.Form):
    name = forms.CharField(max_length=100)
    email = forms.EmailField()
    message = forms.CharField(widget=forms.Textarea)
```

## ModelForm (From Models)

```
from django import forms
from .models import Student

class StudentForm(forms.ModelForm):
    class Meta:
        model = Student
        fields = ['name', 'age', 'email']
```

## Admin Panel

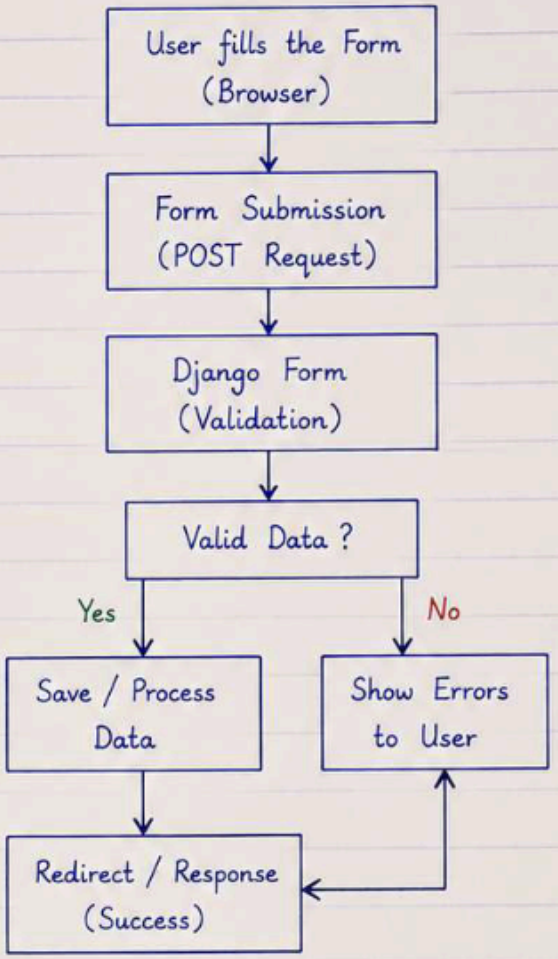
Django provides a built-in admin interface to manage your data easily.

```
python manage.py createsuperuser
```

### Steps

1. Run the command to create superuser.
2. Open /admin in browser.
3. Login with superuser.
4. Manage data (Add, Edit, Delete).

## Form Processing Flow




## Register Model in Admin

```
from django.contrib import admin
from .models import Student

@admin.register(Student)
class StudentAdmin(admin.ModelAdmin):
    list_display = ['id', 'name', 'email']
    search_fields = ['name', 'email']
```

# 7. Authentication

## What is Authentication ?

Authentication is the process of verifying the identity of a user.

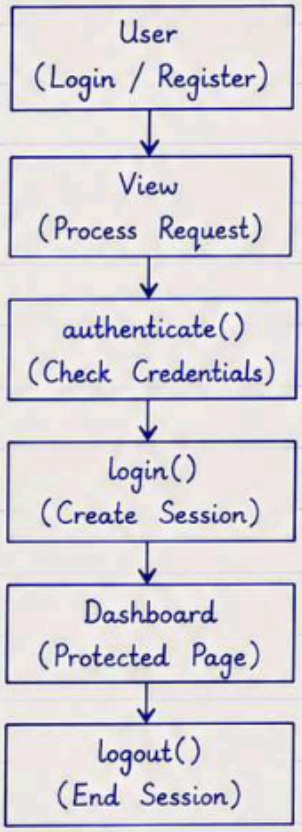
## Key Features

- Register new users
- Login existing users
- Logout users
- Protect views (Login Required)

## Built-in Functions

| | |
|---|---|
| `authenticate()` | – Verifies user credentials |
| `login()` | – Logs in the user |
| `logout()` | – Logs out the user |
| `is_authenticated` | – Checks if user is logged in |

## Authentication Flow




## Example : Login View

```
from django.contrib.auth import authenticate, login
from django.shortcuts import render, redirect

def user_login(request):
    if request.method == 'POST':
        username = request.POST['username']
        password = request.POST['password']
        user = authenticate(request, username=username,
                            password=password)
        if user is not None:
            login(request, user)
            return redirect('home')
        else:
            return render(request, 'login.html', {'error': 'Invalid Credentials'})
    return render(request, 'login.html')
```

## Protecting Views

Use `@login_required` Decorator

```
@login_required
def dashboard(request):
    return render(request, 'dashboard.html')
```

## Steps

1. User sends login request.
2. Credentials are checked using `authenticate()`.
3. If valid, user is logged in using `login()`.
4. User gets access to protected pages.
5. User can logout using `logout()`.

# 8. Django REST Framework

## What is DRF ?

Django REST Framework is a powerful toolkit to build Web APIs in Django easily.

## Key Features

- Serializer for data conversion
- API Views to handle requests
- Authentication & Permissions
- ViewSets & Routers for quick APIs

## Core Components

- Serializer → Converts Model data to JSON
- APIView → Handles API logic
- ViewSet → Group of related APIs
- Router → Automatically creates URLs

## Example : Serializer

```
from rest_framework import serializers
from .models import Student

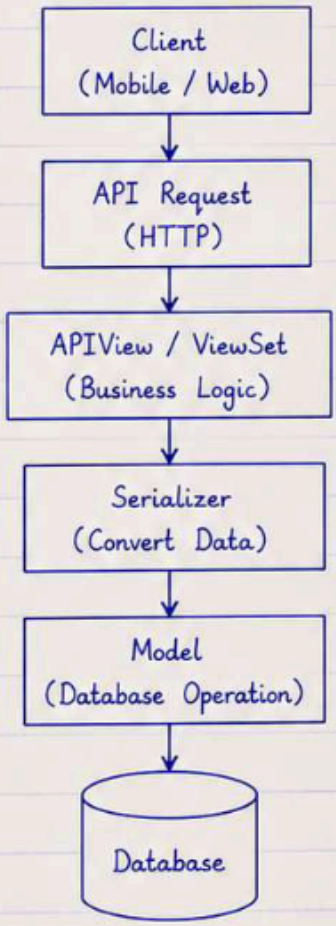
class StudentSerializer(serializers.ModelSerializer):
    class Meta:
        model = Student
        fields = ['id', 'name', 'email', 'age']
```

## Example : Create API using ViewSet

```
from rest_framework import viewsets
from .models import Student
from .serializers import StudentSerializer

class StudentViewSet(viewsets.ModelViewSet):
    queryset = Student.objects.all()
    serializer_class = StudentSerializer
```

## API Flow




## Steps to Create API

1. Create Serializer for the model.
2. Create ViewSet and define queryset & serializer.
3. Register ViewSet with Router.
4. Include router URLs in urls.py.
5. API is ready to use.

# 9. Django Testing

## Why Testing ?

Testing helps to ensure our application works as expected and is bug free.

## Types of Tests

- Unit Tests - Test small parts (functions, methods)
- Integration Tests - Test how parts work together
- Functional Tests - Test functionality from user view

## Writing Tests in Django

- Django provides TestCase class.
- Each test is a method that starts with test_
- Use self.assert* methods to check results.
- Tests are kept in tests.py files.

## Common Assert Methods

- self.assertEqual(a, b) - Check a == b
- self.assertNotEqual(a, b) - Check a != b
- self.assertTrue(x) - Check x is True
- self.assertFalse(x) - Check x is False
- self.assertIn(a, b) - Check a is in b
- self.assertContains(res, text) - Check response contains text

## Running Tests

- Run all tests : python manage.py test
- Run specific app : python manage.py test myapp
- Run specific file : python manage.py test myapp.tests
- Verbose mode : python manage.py test -v 2

# 10. Django Settings & Best Practices

## What is settings.py ?

settings.py is the heart of a Django project. It contains all the configuration and settings for your project.

## Important Settings

- DEBUG – True during development, False in production
- SECRET_KEY – Secret key used for cryptographic signing
- ALLOWED_HOSTS – List of allowed hosts / domains
- INSTALLED_APPS – Apps available in the project
- MIDDLEWARE – Middleware classes to process requests
- ROOT_URLCONF – URL configuration for the project
- TEMPLATES – Template engine configuration
- DATABASES – Database connection settings
- STATIC_URL – URL for static files
- STATIC_ROOT – Root directory for collected static files
- MEDIA_URL – URL for media files
- MEDIA_ROOT – Root directory for media files

## Environment Based Settings

Use environment variables to keep sensitive data secure.

Example:

```
import os
DEBUG = os.environ.get('DEBUG', 'False')
SECRET_KEY = os.environ.get('SECRET_KEY')
DATABASES = {
  'default': {
    'ENGINE': os.environ.get('DB_ENGINE'),
    'NAME': os.environ.get('DB_NAME'),
    'USER': os.environ.get('DB_USER'),
    'PASSWORD': os.environ.get('DB_PASSWORD'),
    'HOST': os.environ.get('DB_HOST'),
    'PORT': os.environ.get('DB_PORT'),
  }
}
```

## Static & Media Files

- Use collectstatic to collect all static files in STATIC_ROOT for production.
- Keep MEDIA files outside the project directory and serve using a web server like Nginx.
- Example:

```
python manage.py collectstatic
STATIC_URL = '/static/'
MEDIA_URL = '/media/'
```

## Best Practices

- Never commit SECRET_KEY or credentials in code. Use .env file.
- Keep DEBUG = False in production.
- Use environment specific settings (dev, test, prod).
- Use a separate database user with limited permissions.
- Regularly backup your database.
- Keep Django and dependencies updated.
- Use HTTPS in production.
- Validate and sanitize user input.

## Common Commands

| | |
|---|---|
| python manage.py runserver | – Run development server |
| python manage.py migrate | – Apply migrations |
| python manage.py makemigrations | – Create new migrations |
| python manage.py createsuperuser | – Create admin user |
| python manage.py collectstatic | – Collect static files |

## Pro Tips

★ Use django-environ or python-decouple to manage environment variables easily.

★ Keep settings.py clean by using separate files for different environments.

★ Good configuration today makes your Django project secure, scalable and production ready !